

5.1 Entity Lifecycle & Persistence Context

Core Concept — Entity States

Persistence Context = First-Level Cache

ASCII

Interview Q / Follow-ups

- Page 1 of 7

5.2 Dirty Checking & Flush

Core Concept

For managed entities, Hibernate keeps a **snapshot** at load time. At flush (before commit or before a query that could be affected), it compares current vs snapshot and auto-generates **UPDATE** for changed fields — **you never call `update()`**.

```
@Transactional
void raise(Long id) {
    Employee e = repo.findById(id).orElseThrow();
    e.setSalary(e.getSalary().multiply(BigDecimal.valueOf(1.1)));
    // no save() needed – dirty checking flushes UPDATE at commit
}
```

FlushMode

AUTO (default): flush before commit and before queries; **COMMIT**: only at commit. `flush()` ≠ `commit()` (flush syncs SQL; commit ends the transaction).

Common Mistakes / Performance

Modifying detached entities expecting auto-update; unnecessary `saveAndFlush` in loops; huge persistence contexts (memory + slow dirty-check) → clear/batch.

Interview Q

- What is dirty checking and when does flush happen?
- Difference between flush and commit.
- Why did an UPDATE fire even though I never called save?

5.3 Caching: First vs Second Level

	L1 (persistence context)	L2 (shared)
Scope	per EntityManager/tx	per SessionFactory (app-wide)
Default	always on	off (opt-in)
Stores	entity instances by id	entity/collection/query data
Provider	built-in	EhCache, Caffeine, Hazelcast, Redis (via provider)
Config	none	<code>@Cacheable</code> (Hibernate), <code>hibernate.cache.use_second_level_cache=true</code>

Query cache is separate and must be enabled explicitly; easy to misuse (invalidation cost). L2 helps read-mostly reference data; risky for write-heavy or multi-node consistency.

Interview Q

- L1 vs L2 cache; when enable L2? Risks with clustering (stale reads / invalidation).

5.4 Lazy vs Eager Loading, Fetch Join & the N+1 Problem

Why Interviewers Ask This

N+1 is the #1 JPA performance bug in production. They want to see you detect and fix it.

Core Concept

- **FetchType.LAZY** — association loaded on first access (proxy). Default for `@OneToMany` / `@ManyToMany`.

- **FetchType.EAGER** — loaded immediately with the owner. Default for `@ManyToOne`/`@OneToOne`. Prefer **LAZY everywhere**; fetch what you need per query.

The N+1 Problem

```
select * from orders;           -- 1 query, returns N orders
for each order: select * from items where order_id=?; -- N queries
=> 1 + N queries
```

Fixes

- **Fetch join** (JPQL): `select o from Order o join fetch o.items` → one query.
- **@EntityGraph** on the repository method (declarative fetch plan).
- **Batch fetching**: `@BatchSize(size=50)` / `hibernate.default_batch_fetch_size` → turns N selects into N/50 `IN (...)` selects.
- **DTO projection**: select only needed columns (avoids entities entirely).

ASCII

```
Bad:  [orders] --then per row--> [items][items][items]... (1+N)
Good: [orders JOIN FETCH items] -> single result set    (1)
```

Real Production Example

An endpoint listing 200 orders each with items ran 201 queries and timed out. Switching to `@EntityGraph(attributePaths="items")` cut it to 1 query; p99 dropped from 3s to 40ms.

Common Mistakes / Trade-offs

- **EAGER everywhere** → loads huge graphs, hidden N+1.
- **join fetch + pagination** → Hibernate paginates in memory (`HHH000104` warning) and can duplicate rows; use `@EntityGraph` or two queries, or fetch ids then entities.
- Multiple **join fetch** on collections → cartesian product.

Debugging

`spring.jpa.show-sql=true` + `hibernate.format_sql`; better: `p6spy` or `datasource-proxy` to count queries; enable `logging.level.org.hibernate.SQL=DEBUG` and bind params `org.hibernate.orm.jdbc.bind=TRACE`. Assert query counts in tests.

Interview Q / Follow-ups

- What is the N+1 problem; how do you detect and fix it?
- LAZY vs EAGER; defaults per association type.
- `join fetch` vs `@EntityGraph`; the pagination pitfall.
- Why prefer DTO projections for read APIs?

Hands-on Exercise

Reproduce N+1 with `@OneToMany` LAZY, confirm 1+N queries in logs, then fix with `@EntityGraph` and verify a single query.

5.5 Cascade Types & Orphan Removal

CascadeType: `PERSIST`, `MERGE`, `REMOVE`, `REFRESH`, `DETACH`, `ALL`. Propagate operations from parent to children. `orphanRemoval=true` deletes children removed from the collection.

Common Mistakes

`CascadeType.ALL` on a `@ManyToOne` to a shared parent (deletes shared data!); confusing `orphanRemoval` with `REMOVE`.

Interview Q

When to use cascade; `orphanRemoval` vs `CascadeType.REMOVE`.

5.6 Transactions (@Transactional)

Core Concept

`@Transactional` (Spring, AOP-proxy based) demarcates a transaction around a method: begin → run → commit, or rollback on unchecked exception.

Key attributes

- **propagation**: `REQUIRED` (default — join or start), `REQUIRES_NEW` (suspend + new), `NESTED` (savepoint), `SUPPORTS`, `MANDATORY`, `NEVER`, `NOT_SUPPORTED`.
- **isolation**: `READ_COMMITTED` (typical default), `REPEATABLE_READ`, `SERIALIZABLE` (see Module 10).
- **rollbackFor**: by default rolls back on `RuntimeException/Error` only — **checked exceptions do NOT roll back** unless `rollbackFor` is set.
- **readOnly=true**: hint (flush mode `MANUAL`, may skip dirty checking) — optimize read paths.
- **timeout**.

Lifecycle / ASCII

```
caller -> [proxy] begin tx (get connection, autocommit=false)
           -> target method runs
           -> commit (or rollback on RuntimeException)
           [proxy] release connection
self-invocation (this.method()) bypasses proxy => NO transaction!
non-public methods => advice not applied (default proxy).
```

Real Production Example

Order + payment must be atomic:

```
@Transactional
public void placeOrder(Order o) {
    orderRepo.save(o);
    paymentClient.charge(o);    // if this throws RuntimeException -> rollback
}
```

For the **outbox pattern**, write the domain change and an outbox row in the same transaction (Module 7).

Common Mistakes (very common in interviews)

- Self-invocation / private / final methods → no transaction.
- Expecting checked exceptions to roll back (they don't by default).
- Calling remote/HTTP inside a long transaction → holds DB connection → pool exhaustion.
- `@Transactional` on the class but a public method calling another public method in the same bean expecting `REQUIRES_NEW` (bypassed).

Interview Q / Follow-ups

- How does `@Transactional` work internally (proxy/AOP)?
- Propagation levels — `REQUIRED` vs `REQUIRES_NEW` vs `NESTED`.
- Which exceptions roll back by default? How to change it?

- Why does self-invocation break it?
- What does `readOnly=true` actually do?

Hands-on Exercise

Demonstrate that a checked exception does NOT roll back, then add `rollbackFor = Exception.class` and observe the rollback.

5.7 JPQL, Criteria API, Specifications, Pagination

- **JPQL** — object-oriented query language over entities: `select o from Order o where o.status = :s. @Query` in repositories; `nativeQuery=true` for raw SQL.
- **Criteria API** — programmatic, type-safe (metamodel) query building; verbose but good for complex dynamic queries.
- **Specifications** (Spring Data) — composable predicates (`spec.and(other)`) for dynamic filters; cleaner than Criteria for search endpoints.
- **Derived queries** — `findByStatusAndCreatedAfter(...)`.
- **Pagination** — `Pageable/Page/Slice`; `Page` runs a count query, `Slice` doesn't (cheaper when you only need "next").

Interview Q

- JPQL vs native SQL vs Criteria vs Specifications — when each?
- `Page` vs `Slice` (extra count query).
- Keyset (seek) pagination vs offset for large tables (offset gets slow deep in).

5.8 Optimistic vs Pessimistic Locking

	Optimistic	Pessimistic
Mechanism	<code>@Version</code> column; check-and-set at commit	DB row lock (<code>SELECT ... FOR UPDATE</code>)
Conflict	<code>OptimisticLockException</code> on stale version	blocks/waits
Best for	low contention, high throughput	high contention, must not lose updates
Cost	retries on conflict	lock contention, deadlock risk
JPA	<code>@Version</code>	<code>@Lock(PESSIMISTIC_WRITE)</code>

Real Production Example

Inventory decrement under concurrency: optimistic `@Version` with retry for most cases; pessimistic `FOR UPDATE` for hot single rows (e.g. a shared counter) to serialize.

Interview Q / Follow-ups

- Optimistic vs pessimistic; how `@Version` works.
- How to handle `OptimisticLockException`? (retry / surface conflict 409.)
- Lost update problem and how each lock prevents it.

Module 5 — One-Page Cheat Sheet

Topic	Key point
Entity states	transient/managed/detached/removed
L1 cache	per-tx persistence context; identity + dirty checking
Dirty checking	snapshot compare at flush → auto UPDATE (no save needed)
flush vs commit	flush syncs SQL; commit ends tx
Fetch	LAZY by default for collections; EAGER for *ToOne — prefer LAZY
N+1	fetch join / @EntityGraph / @BatchSize / DTO projection
join fetch + paging	in-memory paging pitfall (HHH000104) → @EntityGraph
Cascade	ALL/REMOVE/orphanRemoval — beware shared parents
@Transactional	proxy-based; RuntimeException rolls back; self-invocation bypasses
Propagation	REQUIRED vs REQUIRES_NEW vs NESTED
Locking	optimistic @Version (retry) vs pessimistic FOR UPDATE
Pagination	Page(count) vs Slice; keyset for deep pages

Module 5 — Top Interview Questions

1. Explain the persistence context and dirty checking; when does flush happen?
2. What is the N+1 problem — detect and fix (fetch join, EntityGraph, batch size).
3. LAZY vs EAGER; cause of `LazyInitializationException`.
4. How does `@Transactional` work; which exceptions roll back by default?
5. Propagation levels, especially `REQUIRES_NEW` vs `NESTED`.
6. Optimistic vs pessimistic locking; `@Version`.
7. `save` vs `persist` vs `merge`; `Page` vs `Slice`.
8. First vs second level cache; risks of L2 in a cluster.
9. `join fetch` + pagination pitfall.
10. When to use DTO projections / native queries.

Module 5 — Common Mistakes

- EAGER everywhere → hidden N+1 and huge graphs.
- Expecting detached entity changes to persist.
- Remote calls inside a transaction (connection held → pool exhaustion).
- Checked exception not rolling back (default).
- `CascadeType.ALL` deleting shared parents.
- `join fetch` with `Pageable` (in-memory pagination).

Module 5 — Mock Interview

1. “An endpoint runs 201 queries.” → classic N+1; show detection (SQL logs / query counter) and fix with `@EntityGraph`/fetch join/batch size.
2. “`LazyInitializationException` in the JSON serializer.” → lazy access after tx closed; fix with fetch join/EntityGraph or a DTO — NOT `OpenSessionInView` (anti-pattern).

3. *"Two users update inventory concurrently and one loses the update."* → lost update; add `@Version` (optimistic) or `FOR UPDATE` (pessimistic).
4. *"Rollback isn't happening on our checked `PaymentDeclinedException`."* → default only rolls back unchecked; add `rollbackFor`.
5. *"DB connection pool exhausted under load."* → transactions held open across remote calls; shorten transactions, move I/O outside, add timeouts.

Next → Module 6: Spring Security.